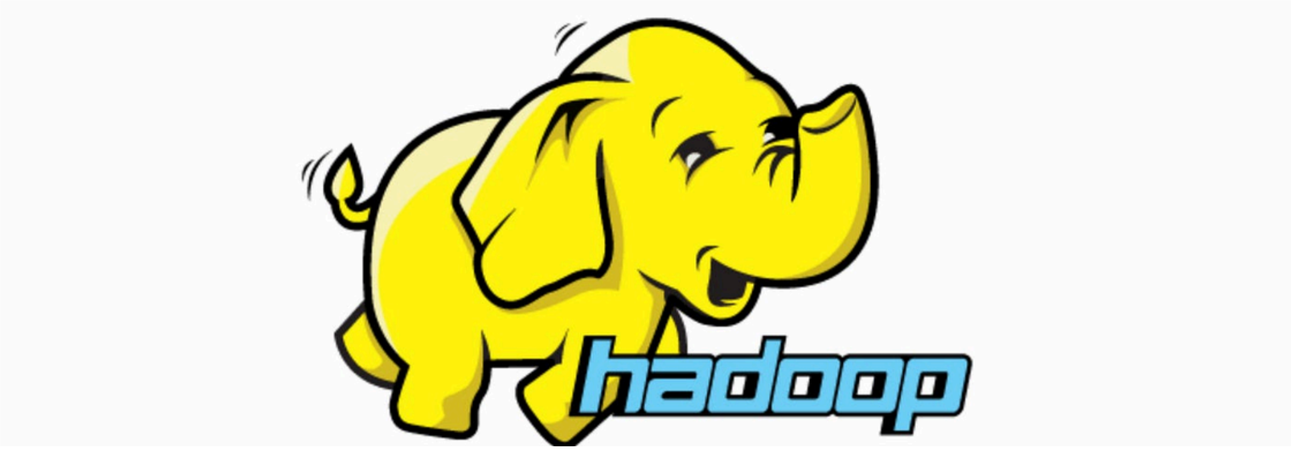


Esercitazione HadOOP

Programmazione

Apache Hadoop è un framework open-source progettato per l'archiviazione e l'elaborazione distribuita di grandi quantità di dati, tipicamente nell'ambito dei Big Data.

Il suo funzionamento si basa sull'utilizzo di un cluster di computer (nodi) che collaborano per gestire dataset troppo grandi o complessi per essere elaborati efficientemente da una singola macchina.



Hadoop permette di suddividere i dati in blocchi, distribuirli tra più nodi e processarli in parallelo, garantendo scalabilità orizzontale e tolleranza ai guasti attraverso meccanismi di replica e recupero.

L'architettura si fonda principalmente su tre componenti: HDFS (Hadoop Distributed File System), responsabile della memorizzazione distribuita dei dati; YARN (Yet Another Resource Negotiator), che gestisce le risorse e l'esecuzione delle applicazioni nel cluster; e MapReduce, un modello di programmazione dedicato all'elaborazione parallela dei dati.

Hadoop rappresenta quindi una delle tecnologie fondamentali dell'ecosistema Big Data, consentendo di gestire grandi volumi di informazioni mediante infrastrutture distribuite.

Esercizio 1 – Analisi di un file di testo con Hadoop

Obiettivo:
Utilizzare Apache Hadoop per eseguire una semplice elaborazione distribuita attraverso HDFS e MapReduce.

Consegna:
Un'azienda possiede un file di testo contenente alcune frasi relative agli acquisti effettuati dai clienti.
L'obiettivo è analizzare il contenuto del file e determinare quante volte compare ciascuna parola utilizzando il classico processo WordCount di Hadoop.

Attività richieste:

1. **Creare un file di testo contenente almeno 10 righe con parole ripetute.**
2. **Creare una directory all'interno di HDFS destinata a contenere i dati.**
3. **Caricare il file di testo dal computer locale al file system distribuito HDFS.**
4. **Verificare che il file sia stato caricato correttamente.**
5. **Eseguire il programma WordCount tramite l'esempio MapReduce disponibile nell'installazione di Hadoop.**
6. **Visualizzare il file di output generato dall'elaborazione.**
7. **Individuare le tre parole che compaiono più frequentemente nel testo.**

Risultato atteso: Al termine dell'esercizio, lo studente dovrà aver compreso come caricare dati su HDFS, eseguire un'elaborazione MapReduce e interpretare i risultati prodotti da Hadoop.

Spiegazione del codice

- **Classe MioMapper: estende la classe Mapper fornita da Hadoop e si occupa della prima fase di elaborazione. Il metodo map() riceve una riga del file, la suddivide attraverso StringTokenizer e genera una coppia chiave-valore per ogni parola. La chiave è la parola stessa, mentre il valore è sempre 1, a indicare una singola occorrenza.**

- **Classe MioReducer: estende la classe Reducer e riceve le parole raggruppate dal framework Hadoop. Per ogni parola, il metodo reduce() scorre i valori associati, esegue la somma e produce una nuova coppia chiave-valore contenente la parola e il numero totale delle sue occorrenze.**

- **Metodo main(): rappresenta il punto di ingresso dell'applicazione. Crea un oggetto Job, configura le classi Mapper e Reducer, definisce i tipi di dati prodotti e specifica i percorsi di input e output. Il metodo waitForCompletion(true) avvia il lavoro e attende la conclusione dell'elaborazione.**

```
import java.io.IOException;
import java.util.StringTokenizer;

import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.IntWritable;
import org.apache.hadoop.io.Text;
import org.apache.hadoop.mapreduce.Job;
import org.apache.hadoop.mapreduce.Mapper;
import org.apache.hadoop.mapreduce.Reducer;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {

  // Classe che analizza le parole del testo
  public static class MioMapper
    extends Mapper<Object, Text, Text, IntWritable> {

    public void map(Object chiave, Text valore, Context context)
        throws IOException, InterruptedException {

      StringTokenizer parole =
        new StringTokenizer(valore.toString());

      while (parole.hasMoreTokens()) {

        String parola = parole.nextToken();

        context.write(
          new Text(parola),
          new IntWritable(1)
        );
      }
    }
  }

  // Classe che conta le occorrenze
  public static class MioReducer
    extends Reducer<Text, IntWritable, Text, IntWritable> {

    public void reduce(Text parola,
              Iterable<IntWritable> valori,
              Context context)
        throws IOException, InterruptedException {

      int totale = 0;

      for (IntWritable numero : valori) {
        totale += numero.get();
      }

      context.write(parola, new IntWritable(totale));
    }
  }

  // Metodo principale
  public static void main(String[] args) throws Exception {

    if (args.length != 2) {
      System.out.println("Uso: WordCount input output");
      return;
    }

    Configuration config = new Configuration();

    Job lavoro = Job.getInstance(config, "Conta Parole");

    lavoro.setJarByClass(WordCount.class);

    lavoro.setMapperClass(MioMapper.class);
    lavoro.setReducerClass(MioReducer.class);

    lavoro.setOutputKeyClass(Text.class);
    lavoro.setOutputValueClass(IntWritable.class);

    FileInputFormat.addInputPath(
      lavoro, new Path(args[0])
    );

    FileOutputFormat.setOutputPath(
      lavoro, new Path(args[1])
    );

    System.exit(lavoro.waitForCompletion(true) ? 0 : 1);
  }
}
```

Esempio pratico di elaborazione - FILE DI INPUT

1. cliente compra pane
2. cliente compra latte
3. cliente compra pane

MAP

Genera una coppia per ogni parola

(cliente, 1)
(compra, 1)
(pane, 1)
(cliente, 1)
(compra, 1)
(latte, 1)
(cliente, 1)
(compra, 1)
(pane, 1)

SHUFFLE E SORT

Raggruppa i valori per parola

cliente → [1, 1, 1]
compra → [1, 1, 1]
pane → [1, 1]
latte → [1]

REDUCE

Somma tutte le occorrenze

cliente 3
compra 3
latte 1
pane 2



Esercizio Hadoop 2 – Analisi delle vendite con MapReduce

In questo esercizio utilizziamo Java e Hadoop MapReduce per analizzare le vendite di un negozio.

A differenza del precedente WordCount, questa volta il programma dovrà leggere un file CSV, estrarre informazioni specifiche e calcolare il totale delle quantità vendute per ciascun prodotto.

1. Dataset di partenza

Creiamo un file chiamato vendite.csv contenente i seguenti dati:

1. **Pane,5**
2. **Latte,3**
3. **Acqua,10**
4. **Pane,2**
5. **Latte,4**
6. **Acqua,5**
7. **Pane,3**
8. **Latte,2**
9. **Acqua,8**
10. **Pane,4**

Ogni riga rappresenta una vendita: il primo valore indica il prodotto, mentre il secondo rappresenta la quantità venduta.

Spiegazione del codice

- **VenditeMapper: legge una riga del file CSV alla volta e utilizza split(",") per separare il nome del prodotto dalla quantità. Il metodo Integer.parseInt() converte la quantità da stringa a numero intero, mentre context.write() produce una coppia chiave-valore contenente il prodotto e la quantità venduta. Le righe con quantità non numeriche vengono ignorate.**

- **VenditeReducer: riceve ogni prodotto insieme alle quantità raggruppate automaticamente da Hadoop. Attraverso un ciclo for, somma tutti i valori ricevuti e restituisce il totale delle unità vendute per quel prodotto.**

- **Metodo main(): configura il lavoro MapReduce, specifica quali classi utilizzare e definisce i percorsi dei dati di ingresso e uscita. Hadoop gestisce automaticamente la fase intermedia di raggruppamento delle coppie chiave-valore.**

```
1.
2. import java.io.IOException;
3.
4. import org.apache.hadoop.conf.Configuration;
5. import org.apache.hadoop.fs.Path;
6. import org.apache.hadoop.io.IntWritable;
7. import org.apache.hadoop.io.Text;
8. import org.apache.hadoop.mapreduce.Job;
9. import org.apache.hadoop.mapreduce.Mapper;
10. import org.apache.hadoop.mapreduce.Reducer;
11. import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
12. import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;
13.
14. public class AnalisiVendite {
15.
16.   // Mapper: legge le vendite
17.   public static class VenditeMapper
18.       extends Mapper<Object, Text, Text, IntWritable> {
19.
20.     public void map(Object key, Text value, Context context)
21.         throws IOException, InterruptedException {
22.
23.       String riga = value.toString();
24.
25.       String[] dati = riga.split(",");
26.
27.       if (dati.length == 2) {
28.
29.         String prodotto = dati[0].trim();
30.         int quantita;
31.
32.         try {
33.           quantita = Integer.parseInt(dati[1].trim());
34.         } catch (NumberFormatException e) {
35.           return;
36.         }
37.
38.         context.write(
39.           new Text(prodotto),
40.           new IntWritable(quantita)
41.         );
42.       }
43.     }
44.   }
45.
46.   // Reducer: calcola il totale venduto
47.   public static class VenditeReducer
48.       extends Reducer<Text, IntWritable, Text, IntWritable> {
49.
50.     public void reduce(Text prodotto,
51.                        Iterable<IntWritable> quantita,
52.                        Context context)
53.         throws IOException, InterruptedException {
54.
55.       int totale = 0;
56.
57.       for (IntWritable valore : quantita) {
58.         totale += valore.get();
59.       }
60.
61.       context.write(
62.         prodotto,
63.         new IntWritable(totale)
64.       );
65.     }
66.   }
67.
68.   public static void main(String[] args) throws Exception {
69.
70.     if (args.length != 2) {
71.       System.out.println("Uso: AnalisiVendite input output");
72.       return;
73.     }
74.
75.     Configuration config = new Configuration();
76.
77.     Job lavoro = Job.getInstance(config, "Analisi Vendite");
78.
79.     lavoro.setJarByClass(AnalisiVendite.class);
80.
81.     lavoro.setMapperClass(VenditeMapper.class);
82.     lavoro.setReducerClass(VenditeReducer.class);
83.
84.     lavoro.setOutputKeyClass(Text.class);
85.     lavoro.setOutputValueClass(IntWritable.class);
86.
87.     FileInputFormat.addInputPath(
88.       lavoro, new Path(args[0])
89.     );
90.
91.     FileOutputFormat.setOutputPath(
92.       lavoro, new Path(args[1])
93.     );
94.
95.     System.exit(lavoro.waitForCompletion(true) ? 0 : 1);
96.   }
97. }
98.
```

Esempio pratico di elaborazione

1.

MAP
Estrazione delle coppie prodotto-quantità
(Pane, 5)
(Latte, 3)
(Pane, 2)
(Acqua, 10)
(Latte, 4)
(Pane, 3)

SHUFFLE E SORT
Raggruppamento per prodotto
Pane → [5, 2, 3, 4]
Latte → [3, 4, 2]
Acqua → [10, 5, 8]

REDUCE
Somma delle quantità per prodotto
14 Pane
9 Latte
23 Acqua




Buon davante a tutti